

STRIPE PROJECT

3. NoSQL Data System

D4. NoSQL Design Diagrams

Nicolas Pichon - AIA RNCP 38777 / BC02 / D4 : NoSQL Design Diagrams / v04 - 2026/10/13.

D4.1 Contexte

Le *business case* demande au le système NoSQL couvre les données semi-structurées et non structurées issues des sources suivantes (cf. [R0] 3. *Data Source / Semi-Structured and Non Structured Data*) :

- interactions avec les utilisateurs (clickstreams et données de session),
- features d'apprentissage automatique (détection de fraude),
- retours clients (avis, réponses d'enquête).
- journaux.

Compte tenu des exigences du *business case* (*TR1*, en particulier), le choix d'un système de stockage *par documents* s'impose (cf. justification en annexe §3.A).

D4.2. Méthode d'analyse

La méthodologie de référence proposée *MongoDB* ([A1] [A2]) consiste d'identifier en premier lieu les motifs d'accès générés par l'application, et d'examiner la forme physique des documents (les schémas) dans un second temps seulement (comme conséquence des motifs d'accès).

D4.3. Principes de conception

Le modèle NoSQL est gouverné par les principes suivant.

1. Modéliser selon les accès

En relationnel, on modélise le domaine puis on écrit les requêtes. En document, on part des requêtes : chaque collection est conçue pour qu'une lecture typique n'implique *aucune jointure*.

2. Schéma flexible mais contrôlé

L'absence de schéma imposé permet d'absorber des structures hétérogènes (un log d'erreur n'a pas les mêmes champs qu'un log d'accès). On applique néanmoins une validation *JSON Schema* sur les champs critiques pour éviter une dérive silencieuse.

3. Imbriquer par défaut, référencer exceptionnellement

Cf. §D4.7. ci-dessous.

D4.4. Analyse des motifs d'accès

Collection	Écriture	Lecture	Ratio écriture/lecture
event_logs	Continue, flux massif	Par fenêtre temporelle	Fortement dominé par l'écriture
user_sessions	Unitaire, à chaque interaction	Session entière, en une fois	Fortement dominé par l'écriture
ml_features	Recalcul périodique par entité	Immédiate par le modèle (inférence temps réel)	Équilibré, lecture critique en latence
customer_feedback	Occasionnelle (après achat/interaction)	Par client, ou agrégée par marchand/produit	Dominé par la lecture agrégée
merchant_config	Quasi jamais	Très fréquente, par clé unique	Quasi exclusivement lu +

Cette analyse commande directement les décisions *embed* / *reference* et les index des sections suivantes (ex: un motif d'accès mal identifié ici produirait un schéma qu'aucun index ne pourrait ensuite corriger efficacement).

D4.5. Collections

D4.5.1 event_logs : journaux applicatifs

Volume très élevé, écriture massive, lecture par fenêtre temporelle.

```
{
  "_id": ObjectId("..."),
  "schema_version": 1,
  "timestamp": ISODate("2026-07-23T14:32:11.482Z"),
  "level": "error",
  "service": "payment-gateway",
  "merchant_id": "a3f1...",
  "transaction_id": "b7c2...",
  "message": "Card authorization timeout",
  "context": {
    "http_status": 504,
    "latency_ms": 30012,
    "retry_count": 2,
    "upstream": "acquirer-eu-1"
  },
  "trace_id": "9f8e7d...",
  "expires_at": ISODate("2026-10-23T14:32:11.482Z")
}
```

Le sous-document `context` est laissé *libre* : sa forme varie selon le service émetteur.

- Clé de *partitionnement* (*shard key*) : { service: 1, timestamp: 1 }
- *Index* : { timestamp: -1 }, { merchant_id: 1, timestamp: -1 }, { transaction_id: 1 } (sparse), { level: 1, timestamp: -1 }
- *TTL* : index sur `expires_at` - purge automatique après 90 jours.

Référencés vs. Embarqués

`merchant_id` et `transaction_id` sont des références d'objets transactionnels;

`context` est une donnée embarquée (toujours lue avec son contexte parent, ici un log d'évènement).

Stratégie de sharding :

- Clé de *sharding* : { service: 1, timestamp: 1 }

Stratégie d'indexation :

- Indexes : { timestamp: -1 }, { merchant_id: 1, timestamp: -1 }, { transaction_id: 1 }, { level: 1, timestamp: -1 }

Durée de vie :

- Index sur `expires_at` + purge automatique après 90 jours (cf. principe de minimisation des données RGPD).

D4.5.2 user_sessions : clickstream et interactions

Une session d'utilisateur = un document. Les événements sont *imbriqués*, car on ne consulte jamais un évènement de clic isolément.

```
{
  "_id": "sess_8f2a...",
  "schema_version": 2,
  "customer_id": "c91d...",
  "merchant_id": "a3f1...",
  "started_at": ISODate("2026-07-23T10:02:00Z"),
  "ended_at": ISODate("2026-07-23T10:19:44Z"),
  "events": [
    {
      "type": "click",
      "x": 100,
      "y": 50,
      "timestamp": ISODate("2026-07-23T10:02:10Z")
    },
    {
      "type": "scroll",
      "x": 100,
      "y": 50,
      "timestamp": ISODate("2026-07-23T10:02:20Z")
    },
    {
      "type": "click",
      "x": 200,
      "y": 100,
      "timestamp": ISODate("2026-07-23T10:02:30Z")
    }
  ]
}
```

```

"duration_sec": 1064,
"event_count": 4,
"device": {
  "type": "mobile",
  "os": "iOS 18.2",
  "browser": "Safari",
  "screen": "390x844"
},
"geo": {
  "ip_country": "FR",
  "region": "Bretagne",
  "city": "Rennes"
},
"events": [
  { "ts": ISODate("2026-07-23T10:02:04Z"), "type": "page_view", "metadata": { "path": "/checkout" } },
  { "ts": ISODate("2026-07-23T10:03:12Z"), "type": "click", "metadata": { "target": "add_card" } },
  { "ts": ISODate("2026-07-23T10:04:55Z"), "type": "form_error", "metadata": { "field": "cvc" } },
  { "ts": ISODate("2026-07-23T10:06:30Z"), "type": "purchase", "metadata": { "transaction_id":
"b7c2..." } }
],
"outcome": "converted"
}

```

Patterns *MongoDB* :

- **Bucket Pattern** :
 - Un document par session, événements embarqués dans un tableau borné par la durée réelle de la session.
- **Polymorphic Pattern** :
 - `metadata` change de forme selon `type` (un événement de porte un `target`, une erreur de formulaire porte un `field`, etc.).
- **Computed Pattern** :
 - `event_count` et `duration_sec` sont pré-calculés à l'écriture pour une relecture directe.
- **Outlier Pattern** :
 - Un document *MongoDB* plafonne à 16 Mo. Une session très longue pourrait dépasser cette taille. On borne donc `events` (par exemple : 1000 entrées) et on bascule le surplus dans un document référencé : `user_sessions_overflow`. C'est le motif dit "*pattern outlier*".

Référencés vs. Embarqués :

- `customer_id` / `merchant_id` référencés.
- `events`, `device`, `geo` embarqués.

Stratégie de sharding :

- Clé de *sharding* : `{ merchant_id: 1, started_at: 1 }` .

Stratégie d'indexation :

- Indexes : `{ customer_id: 1, started_at: -1 }`, `{ "events.transaction_id": 1 }` (multi-key), `{ outcome: 1 }` .

Durée de vie :

- *TTL* : sur `started_at` (90 jours) (une fois la session agrégée vers l'OLAP, la donnée brute n'a pas vocation à être conservée indéfiniment).

D4.5.3. `mL_features` : magasin de features

Alimente la détection de fraude en temps réel. Lecture à très faible latence sur la clé primaire.

une seule lecture doit suffire à disposer de toutes les features nécessaires à la décision, sans agrégation ni jointure à la volée.

Point de conception : un document continuellement actualisé par entité. C'est le bon grain pour un magasin de features servant l'inférence en temps réel : au moment où une nouvelle transaction arrive, il faut récupérer le profil *courant* du client (pas l'historique des évaluations passées). Le magasin de features *NoSQL* est donc un cache d'état courant.

```

{
  "_id": "cust:c91d...",
  "schema_version": 3,
  "entity_type": "customer",
  "entity_id": "c91d...",
  "merchant_id": "a3f1...",
  "computed_at": ISODate("2026-07-23T14:00:00Z"),
  "features": {
    "txn_count_24h": 7,
    "txn_amount_sum_24h": 842.50,
    "distinct_countries_7d": 3,
    "distinct_cards_30d": 2,
    "avg_ticket_90d": 61.30,
    "failed_ratio_7d": 0.21,
    "hours_since_last_txn": 0.4,
    "device_switch_count_24h": 2
  },
  "model_scores": {
    "fraud_v3": 0.87,
    "churn_v1": 0.12
  }
}

```

Patterns *MongoDB*

- **Schema Versioning Pattern** : `schema_version` : l'ensemble de features évolue à chaque itération de modèle sans bloquer les documents déjà écrits avec une version antérieure.
- **Computed Pattern** : les moyennes/compteurs glissants sont pré-calculés par un job périodique (et non pendant l'inférence, où la latence est critique).

Référencés vs. Embarqués

- `entity_id` / `merchant_id` référencés.
- `features` et `model_scores` embarqués (toujours lus ensemble, jamais partiellement).

Stratégie de sharding

- Clé de *sharding* : `{ _id: "hashed" }` (répartition uniforme).

Stratégie d'indexation

- Indexes : `{ merchant_id: 1, computed_at: -1 }`, `{ "model_scores.fraud_v3": -1 }`.

D4.5.4 `customer_feedback` : avis et enquêtes

Volume modeste, structure hétérogène, recherche plein texte.

```

{
  "_id": ObjectId("..."),
  "schema_version": 1,
  "merchant_id": "a3f1...",
  "customer_id": "c91d...",
  "type": "survey_response",
  "submitted_at": ISODate("2026-07-20T09:14:00Z"),
  "channel": "email",
  "content": {
    "rating": 4,
    "text": "Le paiement a fonctionné, mais la confirmation a mis du temps.",
    "language": "fr"
  },
  "context": {
    "order_id": "txn_5a02...",
    "responses": [
      { "question_id": "q1", "answer": "4" },

```

```

    { "question_id": "q2", "answer": "Confirmation lente" }
  ]
},
"nlp": {
  "sentiment": "mixed",
  "sentiment_score": 0.12,
  "topics": ["latency", "confirmation"],
  "model_version": "sent-v2"
}
}

```

Patterns MongoDB

- **Polymorphic Pattern** : un avis noté et une réponse d'enquête NPS partagent la même collection sans partager la même forme de `content`.
- **Attribute Pattern** : les métadonnées variables selon le canal sont regroupées dans `context` plutôt que multipliées en champs racine optionnels.

Référencés vs. Embarqués

- `customer_id` / `merchant_id` / `order_id` référencés (même principe de minimisation que dans les collections précédentes).
- `content`, `context`, `nlp` embarqués.

Stratégie de sharding

- Clé de *sharding* : `merchant_id` ; lecture dominante du document par marchand.

Stratégie d'indexation

- Indexes : `{ merchant_id: 1, submitted_at: -1 }` (tableau de bord marchand), `{ customer_id: 1 }` (historique client, droit d'accès RGPD), index texte sur `content.text` (recherche/analyse de sentiment), `{ "nlp.sentiment": 1 }`

D4.5.5 merchant_config : configuration dénormalisée

Petite collection très lue, quasiment jamais écrite. Cache de référence pour éviter d'interroger les tables transactionnelles à chaque événement applicatif.

```

{
  "_id": "a3f1...",
  "legal_name": "Librairie du Désert",
  "country_code": "FR",
  "status": "active",
  "risk_profile": {
    "tier": "standard",
    "manual_review_threshold": 0.75,
    "auto_block_threshold": 0.95
  },
  "enabled_features": ["3ds", "subscriptions", "instant_payout"],
  "synced_at": ISODate("2026-07-23T06:00:00Z")
}

```

Patterns MongoDB

- **Extended Reference Pattern** : `_id` est la clé de métier transactionnelle (`merchant_id`), mais le document duplique quelques attributs fréquemment lus (`legal_name`, `status`) pour éviter une lecture croisée vers les table transactionnelle à chaque événement. La redondance est acceptable ici précisément parce que ces attributs changent rarement et que `synced_at` documente leur actualité.

Référencés vs. Embarqués

- Cas particulier : seule collection où des attributs normalement référencés (le nom du marchand) sont volontairement dupliqués pour des raisons de performance de lecture.

Stratégie d'indexation

- Indexes : pas besoin d'index de secondaire car on accède directement par `_id` .

D4.6. Stratégies de relations (*)

Le *business case* demande explicitement de traiter l'imbrication, le référencement et l'indexation.
On applique donc les règles suivantes.

Imbrication (embedding)

Retenue quand les données sont lues ensemble, bornées en taille et appartiennent au parent.
Exemples : `events` d'une session, `responses` d'une enquête, ou le sous-document `device` .

Avantage : pas de jointure, une seule lecture disque.
C'est le principal levier de performance en *NoSQL*.

Référencement

Retenu quand l'entité est partagée, volumineuse ou que son cycle de vie est indépendant. Exemples : `transaction_id` , `merchant_id` et `customer_id` référencent les entités transactionnelles.

Point important : ces références sont logiques et ne sont pas des contraintes. Il n'y a pas d'intégrité référentielle garantie par le moteur *NoSQL*. C'est le prix de la flexibilité, et cela impose que la base OLTP reste la source de vérité pour ces entités.

Pattern « extended reference »

Compromis employé dans `event_logs` et `user_sessions` : en plus de la référence (ex: `merchant_id`), on embarque une copie des quelques attributs fréquemment affichés (le nom du marchand, par exemple). On évite ainsi une lecture supplémentaire, au prix d'une redondance acceptable sur des données qui changent rarement (mais qui sont souvent lues).

Indexation

Types d'indexation :

Type	Exemple	
Simple / composé	<code>{ merchant_id: 1, timestamp: -1 }</code>	filtre + tri fréquents
Multikey	<code>{ "events.transaction_id": 1 }</code>	sur un tableau imbriqué
Texte	<code>customer_feedback.text</code>	recherche plein texte
TTL	<code>event_logs.expires_at</code>	purge automatique
Sparse	<code>{ transaction_id: 1 }</code>	le champ n'existe que sur certains logs

Même arbitrage qu'en transactionnel : chaque index accélère la lecture et pénalise l'écriture.
Par exemple, sur le document `event_logs` , accédé intensément en écriture, on minimise volontairement les indexes.

D4.7. Stratégie de *sharding* (synthèse)

Principe directeur : le *cloisonnement par marchand* - déjà appliqué systématiquement dans la base transactionnelle (cf. [D2-C]) - sert de fil conducteur tant que le motif d'accès dominant n'impose pas une autre clé.

Rappel : Le *cloisonnement entre marchands* est le principe selon lequel *les données d'un marchand ne se mélangent jamais avec celles d'un autre* , et ce à tous les niveaux du système. Par exemple, un client qui achète chez trois marchands différents n'est pas, dans le modèle, "une seule personne pour trois achats mais trois enregistrements `Customer` distincts, un par marchand, sans lien direct entre eux dans les données transactionnelles.

Collection	Clé	Justification
<code>event_logs</code>	<code>{ service: 1, timestamp: 1 }</code>	Écriture répartie par service émetteur, lecture par fenêtre temporelle
<code>user_sessions</code>	<code>{ merchant_id: 1, started_at: 1 }</code>	Cloisonnement marchand préservé

Collection	Clé	Justification
ml_features	{ _id: "hashed" }	Répartition uniforme, accès par clé unique sans pattern marchand dominant
customer_feedback	merchant_id	Lecture dominante par marchand (tableau de bord)
merchant_config	Non "shardée"	Volume trop faible pour le justifier

D4.8. Validation de schémas

Chaque collection porte une validation *JSON Schema* minimale : champs obligatoires et types de base, qui ne fige pas la forme des champs variables (context , features , metadata).

Exemple avec user_sessions :

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["schema_version", "customer_id", "merchant_id", "started_at"],
    "properties": {
      "schema_version": { "bsonType": "int" },
      "customer_id": { "bsonType": "string" },
      "merchant_id": { "bsonType": "string" },
      "started_at": { "bsonType": "date" }
    }
  }
}
```

Le tableau events et ses metadata internes restent hors validation, volontairement, pour ne pas réintroduire la rigidité qu'on cherche à éviter.

D4.9. Intégration OLTP / OLAP

Le business case exige que le modèle NoSQL s'intègre aux deux autres systèmes.

OLTP > NoSQL

Les captures CDC sur transaction , merchant et customer alimentent un flux Kafka (cf. [D1]). Un consommateur met à jour merchant_config et déclenche le recalcul des ml_features . Les identifiants OLTP (transaction_id , merchant_id) servent de clés de rapprochement.

NoSQL > OLAP

Les sessions et les scores de fraude sont agrégés par lots nocturnes et chargés dans FactFraudEvent et les dimensions comportementales. Les données brutes restent dans le système NoSQL , seul l'agrégat part vers l'entrepôt.

NoSQL comme couche de rapprochement transverse

C'est dans le système NoSQL que l'on peut rapprocher une même personne physique agissant chez plusieurs marchands (via l'empreinte de sa carte), sans compromettre le cloisonnement transactionnel entre marchands.

D4.10. Cohérence, sécurité et conformité

Modèle de cohérence

La vérification de cohérence à terme (*eventual consistency*) est acceptable pour les journaux et les sessions (un délai de quelques secondes sera sans conséquence). Inversement, sur le chemin de décision de fraude, où une donnée périmée a un coût réel, ml_features est lu avec la garantie readConcern: "majority" .

readConcern est un réglage MongoDB qui définit la garantie de fraîcheur et de durabilité des données lues. "majority" est le niveau le plus strict couramment utilisé : il garantit que la donnée lue a été confirmée par la majorité des membres du replica set (l'ensemble des répliques d'un même jeu de données), et qu'elle ne sera donc pas annulée par un rollback ultérieur.

Cette garantie a un coût : la lecture est plus lente qu'une lecture sur un seul nœud, puisqu'il faut attendre la confirmation de plusieurs répliques plutôt que d'en interroger une seule immédiatement disponible.

Chiffrement

Chiffrement au repos (au niveau du stockage) et en transit (TLS). Les champs sensibles (`text` des retours clients, données d'identification) relèvent du chiffrement au niveau du champ (*Client-Side Field Level Encryption*).

RGPD

Le droit à l'effacement impose de pouvoir supprimer toutes les traces d'un client. Les indexes sur `customer_id` présents dans chaque collection rendent cette opération réalisable. Les indexes *TTL* assurent par ailleurs la limitation de conservation des journaux.

PCI-DSS

Aucune donnée de carte complète n'est stockée dans le système *NoSQL*, conformément au principe déjà appliqué en *OLTP*.

Contrôle d'accès

Rôles définis par collection (lecture seule sur `event_logs` pour les analystes, écriture réservée aux services), et journalisation des accès.

D4.11. Couverture des exigences

#	Exigence	Couverture
BR3	schéma flexible, types de données variés, requêtage de données imbriquées	Cinq collections, patterns Polymorphic/Attribute (§D4.5)
TR1	schéma NoSQL flexible et requêtable	Index ciblés (§D4.6), validation minimale non bloquante (§D4.8)
TR3	bases distribuées, sharding	Clé de sharding justifiée par collection (§D4.7)

Références

Ressources

- [R0] [Stripe Business Case](#)

Livrables

- [D1] [Data Architecture Diagram](#)

Annexes

- [D2-B] [OLTP Physical Model](#)
- [D2-C] [OLTP Supporting Notes](#)

Applicables

- [A1] [MongoDB : Designing Your Schema](#)
- [A2] [MongoDB : Schema Design Patterns](#)

Annexes

3.A. Justification du modèle de stockage

Pourquoi choisir une base orientée *documents*?

La première exigence technique du *business case* (TR1) demande explicitement un système capable de requêter efficacement des données non structurées imbriquées ("*efficient querying of nested and unstructured data*"). Cette exigence précise élimine les trois autres familles de stockage *NoSQL* (cf. analyse ci-dessous).

Sources de données

D'après le *business case*, les types de données que le système *NoSQL* prendre en compte sont les suivants (cf. 3. *Data Source/ Semi-Structured and Non Structured Data*) :

- *Clickstreams des utilisateurs & données de sessions* : un événement de clic n'a pas les mêmes attributs qu'une soumission de formulaire ou qu'une vue de page ; la structure varie par nature d'un enregistrement à l'autre.
- *Machine Learning features (détection de fraude)* : un vecteur de caractéristiques évolue au fil des itérations du modèle : on ajoute, retire, recombine des variables sans prévenir.
- *Retours clients (avis, enquêtes)* : texte libre plus métadonnées dont les champs varient selon le canal de collecte.

Le point commun de ces sources : une structure imprévisible et changeante, et de gros volumes.

Pourquoi rejeter les trois autres familles de stockage?

Stockage par clé-valeur (*Redis, DynamoDB*)

Excellent pour une lecture par clé unique, mais aucune capacité native à interroger à l'intérieur de la donnée. Impossible de requêter "*tous les événements t.q. device_type = mobile et event_type = click*" sans tout rapatrier du côté de l'application. Or c'est exactement ce type de requête qu'exige l'exigence TR1.

Stockage par colonne (*Cassandra, HBase*)

Très bon pour un débit d'écriture extrême sur un schéma de colonnes relativement stable, mais une famille de colonnes reste rigide par nature. Mal adapté à des enregistrements dont la forme change d'un type d'événement à l'autre.

Stockage en graphe (*Neo4j*)

Pertinent pour des requêtes de traversée de relations (détection de réseaux de fraude coordonnée, par exemple), mais ce n'est pas le besoin ici : on stocke des événements et des vecteurs de caractéristiques, pas un graphe de relations à parcourir.

Ce que le stockage par documents apporte spécifiquement

Le format correspond déjà à la donnée source

Un événement de clickstream envoyé par une app web/mobile est nativement un objet JSON. Le stocker tel quel, sans le forcer dans un schéma relationnel fixe, évite une transformation qui n'apporterait rien.

Schéma flexible sans migration

Un nouveau type d'événement, un nouveau champ de feature ML : un document orienté document l'absorbe immédiatement, sans `ALTER TABLE` ni migration des enregistrements existants. Cohérent avec un système qui, par nature, évolue en continu (nouvelles features testées, nouvelle instrumentation produit).

Requêtage riche sur des champs imbriqués

Indexation secondaire sur des sous-champs, agrégations sur des tableaux imbriqués. C'est exactement ce que demande l'exigence TR1 (et ce qu'un simple stockage clé-valeur ne permet pas).

Passerelle naturelle vers le pipeline déjà conçu

Le choix d'une base orientée documents s'imbrique bien dans l'architecture globale posée en D1 : le mécanisme des *Change Streams* propre aux systèmes de stockage par document (équivalent au mécanisme *CDC* dans un système transactionnel) permettent d'alimenter le pipeline *Airflow* → *OLAP* par le même principe de réplication asynchrone qui alimente `ChangeAuditLog`.

Sharding horizontal natif

Répond à TR3 ("*distributed databases and data sharding*") - un partitionnement par `customer_id` ou `session_id` scale horizontalement sans repenser le modèle.
